



Sensori e Localizzazione

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet
2025/2026

Il framework dei sensori Android

Android fornisce accesso a sensori hardware tramite il SensorManager. Ogni sensore produce eventi con valori numerici a intervalli regolari.



Sensori di movimento

Accelerometro, giroscopio, vettore di rotazione. Misurano forze e rotazioni.



Sensori di posizione

Magnetometro (bussola), prossimità. Orientamento e distanza da oggetti.



Sensori ambientali

Luce, pressione barometrica, temperatura, umidità. Condizioni dell'ambiente.



Non tutti i dispositivi hanno tutti i sensori. Verifica sempre la disponibilità prima di registrarti.

Elencare i sensori disponibili

Il `SensorManager` permette di scoprire quali sensori sono presenti sul dispositivo.

```
val sensorManager = getSystemService(Context.SENSOR_SERVICE) as SensorManager

// Lista tutti i sensori
val allSensors: List<Sensor> = sensorManager.getSensorList(Sensor.TYPE_ALL)
allSensors.forEach { sensor ->
    Log.d("Sensors", "${sensor.name} - tipo: ${sensor.type}")
}

// Verificare se un sensore specifico esiste
val accelerometer: Sensor? = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
if (accelerometer != null) {
    Log.d("Sensors", "Accelerometro disponibile: ${accelerometer.name}")
} else {
    Log.w("Sensors", "Accelerometro NON disponibile su questo dispositivo")
}
```

Kotlin



`getDefaultSensor()` ritorna null se il sensore non esiste. Controllare SEMPRE prima di registrarli.

Registrare un SensorEventListener

Per ricevere dati da un sensore si implementa `SensorEventListener` e lo si registra nel ciclo di vita dell'Activity.

```
class SensorActivity : AppCompatActivity(), SensorEventListener {  
    private lateinit var sensorManager: SensorManager  
    private var accelerometer: Sensor? = null  
  
    override fun onCreate(b: Bundle?) {  
        super.onCreate(b)  
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager  
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)  
    }  
  
    override fun onResume() {  
        super.onResume()  
        accelerometer?.let {  
            sensorManager.registerListener(this, it, SensorManager.SENSOR_DELAY_UI)  
        }  
    }  
  
    override fun onPause() {  
        super.onPause()  
        sensorManager.unregisterListener(this) // SEMPRE!  
    }  
  
    override fun onSensorChanged(event: SensorEvent) { /* dati */ }  
    override fun onAccuracyChanged(sensor: Sensor, accuracy: Int) { }  
}
```

Kotlin

Frequenza di campionamento

Il terzo parametro di `registerListener` controlla la frequenza degli eventi. Più veloce = più batteria consumata.

Costante	Intervallo	Caso d'uso
<code>SENSOR_DELAY_FASTEST</code>	0 μ s (più veloce possibile)	Acquisizione dati scientifici
<code>SENSOR_DELAY_GAME</code>	20.000 μ s (~50 Hz)	Giochi, realtà aumentata
<code>SENSOR_DELAY_UI</code>	60.000 μ s (~16 Hz)	Aggiornamento interfaccia utente
<code>SENSOR_DELAY_NORMAL</code>	200.000 μ s (~5 Hz)	Rotazione schermo, bussola



Impatto sulla batteria

`SENSOR_DELAY_FASTEST` consuma molta batteria. Usare `SENSOR_DELAY_UI` o `NORMAL` salvo reale necessità.



Il delay è un suggerimento al sistema: la frequenza effettiva può variare a seconda dell'hardware.

Leggere i dati: onSensorChanged

Ogni evento contiene un array `values[]` con i dati del sensore. Il significato dipende dal tipo.

```
override fun onSensorChanged(event: SensorEvent) {  
    when (event.sensor.type) {  
        Sensor.TYPE_ACCELEROMETER -> {  
            val x = event.values[0] // m/s2  
            val y = event.values[1]  
            val z = event.values[2]  
            binding.accelText.text =  
                "X=%.2f Y=%.2f Z=%.2f".format(x, y, z)  
        }  
        Sensor.TYPE_LIGHT -> {  
            val lux = event.values[0]  
            binding.lightText.text =  
                "Luce: %.0f lux".format(lux)  
        }  
        Sensor.TYPE_PROXIMITY -> {  
            val cm = event.values[0]  
            binding.proxText.text =  
                if (cm < 5) "Vicino" else "Lontano"  
        }  
    }  
}
```

Kotlin

SensorEvent.values[]

Accelerometro

[0]=X, [1]=Y, [2]=Z (m/s²)

Giroscopio

[0]=X, [1]=Y, [2]=Z (rad/s)

Luce

[0]=lux

Prossimità

[0]=distanza in cm

Pressione

[0]=hPa (millibar)

Magnetometro

[0]=X, [1]=Y, [2]=Z (μT)

Esempio: rilevare uno shake

Combinando i valori dell'accelerometro possiamo rilevare un gesto di "scuotimento" del dispositivo.

```
private var lastShakeTime = 0L

override fun onSensorChanged(event: SensorEvent) {
    if (event.sensor.type != Sensor.TYPE_ACCELEROMETER) return

    val x = event.values[0]
    val y = event.values[1]
    val z = event.values[2]

    // Calcola la forza totale (sottrai la gravita ~9.8)
    val gForce = Math.sqrt(
        (x * x + y * y + z * z).toDouble()
    ) / SensorManager.GRAVITY_EARTH

    if (gForce > 2.5) { // soglia di shake
        val now = System.currentTimeMillis()
        if (now - lastShakeTime > 500) { // debounce 500ms
            lastShakeTime = now
            onShakeDetected()
        }
    }
}

private fun onShakeDetected() {
    Toast.makeText(this, "Shake rilevato!", Toast.LENGTH_SHORT).show()
}
```

Kotlin

Esempio: bussola digitale

Combinando accelerometro e magnetometro si ottiene l'orientamento del dispositivo (heading).

```
private val gravity = FloatArray(3)
private val geomagnetic = FloatArray(3)

override fun onSensorChanged(event: SensorEvent) {
    when (event.sensor.type) {
        Sensor.TYPE_ACCELEROMETER -> event.values.copyInto(gravity)
        Sensor.TYPE_MAGNETIC_FIELD -> event.values.copyInto(geomagnetic)
    }

    val rotationMatrix = FloatArray(9)
    val orientation = FloatArray(3)

    if (SensorManager.getRotationMatrix(rotationMatrix, null, gravity, geomagnetic)) {
        SensorManager.getOrientation(rotationMatrix, orientation)
        val azimuth = Math.toDegrees(orientation[0].toDouble()).toFloat()
        val heading = (azimuth + 360) % 360 // 0-360 gradi

        binding.compassText.text = "%.0f°".format(heading)
        binding.compassImage.rotation = -heading // ruota l'icona
    }
}
```

Kotlin



Registrare ENTRAMBI i sensori (accelerometro + magnetometro) per ottenere la rotazione.

Localizzazione: panoramica

Android offre più fonti per determinare la posizione del dispositivo. Google Play Services le unifica in FusedLocationProvider.



GPS (Satellite)

Alta precisione (~3m), lento al primo fix, consuma batteria, non funziona al chiuso.



Rete (Wi-Fi + Cell)

Precisione media (~30-100m), veloce, funziona al chiuso, basso consumo.



Fused Location Provider

Combina GPS + rete + sensori. Sceglie automaticamente la fonte migliore per rispettare precisione e batteria.



Usare SEMPRE FusedLocationProviderClient (Google Play Services): gestisce la complessità per te.

Permessi di localizzazione

La localizzazione è un dato sensibile. Servono permessi a runtime con livelli crescenti di intrusività.

Permesso	Descrizione
ACCESS_COARSE_LOCATION	Posizione approssimata (~100m, basata sulla rete). API 1+.
ACCESS_FINE_LOCATION	Posizione precisa (~3m, GPS). Include anche COARSE. API 1+.
ACCESS_BACKGROUND_LOCATION	Posizione anche quando l'app è in background. API 29+. Richiesta separata.

```
<!-- AndroidManifest.xml -->  
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>  
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
```

XML

**Android 12+ (API 31): scelta dell'utente**

L'utente può concedere solo la posizione approssimata anche se si chiede FINE_LOCATION. Gestire entrambi i casi.



ACCESS_BACKGROUND_LOCATION va richiesto DOPO aver ottenuto FINE o COARSE, in una richiesta separata.

Richiesta permessi a runtime

Usa `ActivityResultContracts` per richiedere i permessi di localizzazione.

```
private val locationLauncher = registerForActivityResult(  
    ActivityResultContracts.RequestMultiplePermissions()  
) { permissions ->  
    val fineGranted = permissions[Manifest.permission.ACCESS_FINE_LOCATION] ?: false  
    val coarseGranted = permissions[Manifest.permission.ACCESS_COARSE_LOCATION] ?: false  
  
    when {  
        fineGranted -> startLocationUpdates(Priority.PRIORITY_HIGH_ACCURACY)  
        coarseGranted -> startLocationUpdates(Priority.PRIORITY_BALANCED_POWER_ACCURACY)  
        else -> showPermissionDeniedMessage()  
    }  
}  
  
private fun requestLocationPermission() {  
    locationLauncher.launch(arrayOf(  
        Manifest.permission.ACCESS_FINE_LOCATION,  
        Manifest.permission.ACCESS_COARSE_LOCATION  
    ))  
}
```

Kotlin

FusedLocationProviderClient

Il client unificato di Google Play Services per ottenere la posizione. Gestisce GPS, Wi-Fi e sensori automaticamente.

```
// Dipendenza: implementation("com.google.android.gms:play-services-location:21.1.0")
```

Kotlin

```
private lateinit var fusedClient: FusedLocationProviderClient

override fun onCreate(b: Bundle?) {
    super.onCreate(b)
    fusedClient = LocationServices.getFusedLocationProviderClient(this)
}

// Ultima posizione nota (veloce, puo essere null)
@SuppressLint("MissingPermission")
private fun getLastLocation() {
    fusedClient.lastLocation.addOnSuccessListener { location: Location? ->
        if (location != null) {
            val lat = location.latitude
            val lng = location.longitude
            binding.locationText.text = "Lat: $lat, Lng: $lng"
        } else {
            binding.locationText.text = "Posizione non disponibile"
        }
    }
}
```

Aggiornamenti continui della posizione

Per tracciare il movimento si richiedono aggiornamenti periodici con un `LocationRequest`.

```
private val locationRequest = LocationRequest.Builder(  
    Priority.PRIORITY_HIGH_ACCURACY, 10_000L // intervallo: 10 secondi  
)  
.apply {  
    setMinUpdateIntervalMillis(5_000L) // minimo: 5 secondi  
    setMinUpdateDistanceMeters(10f) // minimo: 10 metri  
}.build()  
  
private val locationCallback = object : LocationCallback() {  
    override fun onLocationResult(result: LocationResult) {  
        val location = result.lastLocation ?: return  
        binding.latText.text = "Lat: ${location.latitude}"  
        binding.lngText.text = "Lng: ${location.longitude}"  
        binding.accText.text = "Precisione: ${location.accuracy}m"  
    }  
}  
  
// Avviare gli aggiornamenti (dopo il controllo permessi)  
@SuppressWarnings("MissingPermission")  
fun startLocationUpdates() {  
    fusedClient.requestLocationUpdates(locationRequest, locationCallback, Looper.getMainLooper())  
}  
  
// Fermare gli aggiornamenti (in onPause o quando non serve piu)  
fun stopLocationUpdates() = fusedClient.removeLocationUpdates(locationCallback)
```

Kotlin

Priority: precisione vs batteria

Il parametro Priority bilancia precisione e consumo energetico.

Priority	Fonti	Precisione	Batteria	Caso d'uso
PRIORITY_HIGH_ACCURACY	GPS + rete + sensori	~3 m	Alto	Navigazione, fitness
PRIORITY_BALANCED_POWER_ACCURACY	Rete (Wi-Fi + cell)	~100 m	Medio	Mostrare la città
PRIORITY_LOW_POWER	Solo celle	~10 km	Basso	Geofencing ampio
PRIORITY_PASSIVE	Piggyback su altre app	Variabile	Minimo	Logging opportunistico



Regola d'oro: usare la precisione PIÙ BASSA che soddisfa il caso d'uso. Meno precisione = meno batteria.

Localizzazione in Compose

In un'app Compose i permessi si richiedono con `rememberLauncherForActivityResult` e la posizione si raccoglie in uno stato.

```
@Composable
fun LocationScreen() {
    val context = LocalContext.current
    var location by remember { mutableStateOf<Location?>(null) }

    val launcher = rememberLauncherForActivityResult(
        ActivityResultContracts.RequestMultiplePermissions()
    ) { perms ->
        if (perms[Manifest.permission.ACCESS_FINE_LOCATION] == true) {
            val client = LocationServices.getFusedLocationProviderClient(context)
            client.lastLocation.addOnSuccessListener { location = it }
        }
    }

    Column(Modifier.fillMaxSize().padding(16.dp)) {
        Button(onClick = {
            launcher.launch(arrayOf(
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            ))
        }) { Text("Ottieni posizione") }

        location?.let { loc ->
            Text("Lat: ${loc.latitude}, Lng: ${loc.longitude}")
        }
    }
}
```

Kotlin

Geofencing

Un geofence è un'area circolare virtuale sulla mappa. Android notifica l'app quando l'utente entra, esce o sosta nell'area.

```
val geofence = Geofence.Builder()
    .setRequestId("universita")
    .setCircularRegion(45.4642, 11.3523, 200f) // lat, lng, raggio in metri
    .setTransitionTypes(
        Geofence.GEOFENCE_TRANSITION_ENTER or Geofence.GEOFENCE_TRANSITION_EXIT
    )
    .setExpirationDuration(Geofence.NEVER_EXPIRE)
    .build()

val request = GeofencingRequest.Builder()
    .setInitialTrigger(GeofencingRequest.INITIAL_TRIGGER_ENTER)
    .addGeofence(geofence)
    .build()

val pendingIntent = PendingIntent.getBroadcast(
    context, 0, Intent(context, GeofenceReceiver::class.java),
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_MUTABLE
)

geofencingClient.addGeofences(request, pendingIntent)
```

Kotlin

Best practice

DA FARE

- Unregister sensori in onPause()
- Verificare disponibilità sensore (null check)
- Usare SENSOR_DELAY_UI se basta
- FusedLocationProvider, non LocationManager
- Precisione più bassa possibile
- Fermare gli aggiornamenti quando non servono
- Spiegare all'utente perché serve la posizione

DA EVITARE

- Dimenticare unregisterListener()
- SENSOR_DELAY_FASTEST senza motivo
- HIGH_ACCURACY per mostrare la città
- Richiedere BACKGROUND_LOCATION subito
- Ignorare il caso "permesso negato"
- LocationManager diretto (legacy)
- Aggiornamenti continui con app in background

Errori comuni e soluzioni

Sensore non disponibile → crash

`getDefaultSensor()` ritorna null. Controlla SEMPRE con `?.` o `if != null`.

Posizione sempre null

`lastLocation` è null se nessuna app ha richiesto la posizione di recente. Usa `requestLocationUpdates`.

Posizione imprecisa al chiuso

GPS non funziona indoor. Usa `BALANCED_POWER` che sfrutta Wi-Fi e celle.

Batteria scarica rapidamente

Sensore registrato in `onCreate` e non de-registrato in `onPause`. O `DELAY_FASTEST`.

SecurityException su location

Permesso non richiesto a runtime. Verifica con `checkSelfPermission` prima di ogni chiamata.

Geofence non si attiva

Richiede `ACCESS_BACKGROUND_LOCATION` su API 29+. Verifica anche la batteria e Doze mode.

Cheat-sheet

Kotlin

```
// 1. Sensori: ottenere il manager
val sm = getSystemService(SENSOR_SERVICE) as SensorManager
val accel = sm.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

// 2. Registrare / de-registrare
sm.registerListener(listener, accel, SensorManager.SENSOR_DELAY_UI) // onResume
sm.unregisterListener(listener)                                     // onPause

// 3. Leggere i dati
override fun onSensorChanged(e: SensorEvent) { val x = e.values[0] }

// 4. Localizzazione: FusedLocationProviderClient
val client = LocationServices.getFusedLocationProviderClient(this)
client.lastLocation.addOnSuccessListener { loc -> /* lat, lng */ }

// 5. Aggiornamenti continui
val req = LocationRequest.Builder(Priority.PRIORITY_HIGH_ACCURACY, 10_000L).build()
client.requestLocationUpdates(req, callback, Looper.getMainLooper())
client.removeLocationUpdates(callback)

// 6. Permessi (Manifest)
// ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION, ACCESS_BACKGROUND_LOCATION

// 7. Geofence
Geofence.Builder().setCircularRegion(lat, lng, radius).build()
```